

Other Graph Algorithms and Proofs

This chapter is an overview of other graph algorithms, proofs, and concepts that are important to know. We will first cover adjacency matrices, then Prim's algorithm for minimum spanning trees, isomorphism, and topological sorting for directed acyclic graphs (DAGs).

1.1 Adjacency Matrices

Recall that we can represent graphs as lists of data called adjacency lists. In code, this could look like the following:

- A: [B, C]
- B: [A, D]
- C: [A]
- D: [B]

This represents the following graph

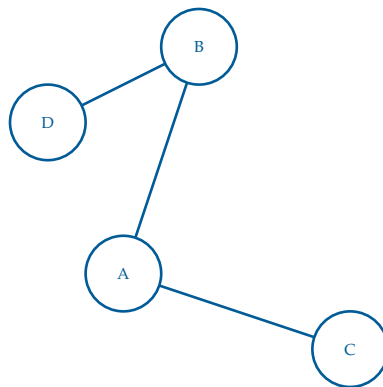


Figure 1.1: A simple (undirected unweighted) graph used to make an adjacency list and adjacency matrix.

We could also represent the same data as a *matrix*, which we have covered in linear algebra. The *adjacency matrix* is 2D array (as compared to an adjacency list, which consists of

multiple 1D arrays) where $\text{matrix}[i][j]$ indicates whether an edge exists from vertex i to vertex j . Commonly, $\text{matrix}[i][j] = 1$ if and only if the edge from vertex i to vertex j exists, else $\text{matrix}[i][j] = 0$.¹ The above graph in Figure 1.1 can be represented as:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

For *undirected* graphs, the adjacency matrix is symmetric, and thusly $n \times n$, meaning that the transpose of matrix is the same matrix!

$$A = A^T$$

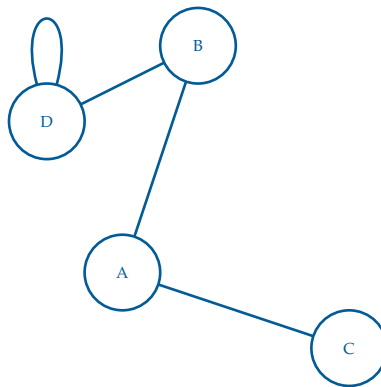


Figure 1.2: A simple graph with an added loop on vertex D.

By convention, a diagonal entry such as $\text{matrix}[k][k]$, where $(k \in \mathbb{Z})$, is non-zero only if there is a loop from a vertex to itself. The value of the entry represents the number of such loops. In this case, there are two edges from vertex (D) to itself in Figure 1.2, so the corresponding diagonal entry is (2).

For *directed* graphs, the adjacency matrix appears a bit different. A *directed adjacency matrix* represents a directed graph, where edges have a direction (arrows). When $A_{ij} = 1$, then an edge exists from vertex i to vertex j .

¹If the edges are weighted, the $\text{matrix}[i][j]$ is equal to the weight, or 0 if the edge does not exist.

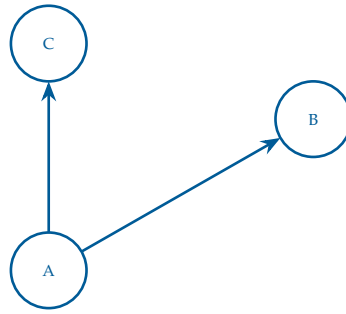


Figure 1.3: A simple directed graph used to make an adjacency matrix.

The resulting adjacency matrix is

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Notice how, now, $A \neq A^T$, because the edges are directional.

Exercise 1 Adjacency Matrix Exercise

Working Space

The following is an adjacency matrix for a weighted (undirected) graph.

	A	B	C	D	E	F	G
A	0	4	2	0	0	0	0
B	4	0	1	5	0	0	0
C	2	1	0	8	10	0	0
D	0	5	8	0	2	6	0
E	0	0	10	2	0	3	7
F	0	0	0	6	3	0	4
G	0	0	0	0	7	4	0

Draw the corresponding graph with weighted edges.

Answer on Page 7

1.2 Prim's Algorithm and Kruskal's Algorithm for Minimum Spanning Trees

Prim's Algorithm is minimum spanning tree

FIXME <https://github.com/KontinuaFoundation/sequence/issues/315>

1.3 A* (A-star) Search Algorithm

The last shortest path we will discuss in this chapter is the *A-star Algorithm*, more commonly represented as *A* Algorithm*. In essence, it is an extension of Dijkstra's Algorithm in combination with a *priority-based heuristic function*. A heuristic is an introduced estimated cost from any node to the goal.

The key behind this implementation is the following function

$$f(n) = g(n) + h(n)$$

where

- $g(n)$: the cost of the path from the start node (usually S) to the current node n
- $h(n)$: the *heuristic* function that estimates the remaining cost from n to the goal (usually E)
- $f(n) = g(n) + h(n)$: the estimated total cost of a path from S to E passing through node n

1.3.1 The Path Cost $g(n)$

$g(n)$ remains to be the same value that we used in Dijkstra's and other shortest path algorithms.

When moving from a node u to an adjacent node v , the path cost is *updated* as

$$g(v) = g(u) + c(u, v)$$

where $c(u, v)$ is the cost of traversing the edge from u to v . As we discussed in the chapter on Dijkstra's, we have a decision to make upon each visit to a node: Each time we visit a node, we have a decision to make:

$$\text{if } \text{dist}[u] + w(u, v) < \text{dist}[v] \text{ then update } \text{dist}[v]$$

1.3.2 The Heuristic Function $h(n)$

The heuristic $h(n)$ function depends on the problem domain. Its purpose is to estimate the remaining cost from node n to the goal. While this is a difficult task, we can use basic algebra to make an informed guess.

For example, in a map-based pathfinding problem, $h(n)$ may be the straight-line distance, or *Euclidean distance* between node n and the goal. Since the actual path cannot be shorter than the Euclidean distance, this provides a reasonable estimate of the remaining cost. If a node's location is represented on a 2D plane (such as a map or graph) (x_n, y_n) and the goal location as (x_g, y_g) , the Euclidean Heuristic

$$h(n) = \sqrt{(x_g - x_n)^2 + (y_g - y_n)^2}$$

Alternatively, many 2D games use a similar either *taxicab distance* (typically for 4-way movement),

$$h(n) = |x_g - x_n| + |y_g - y_n|$$

or *Chebyshev distance* (typically for 8-way movement in a game)

$$d = \max(|x_2 - x_1|, |y_2 - y_1|)$$

which takes the maximum of the horizontal and vertical movements.

Sometimes, graphs without direct spatial meaning become more difficult to work with because there is no obvious notion of distance between two nodes.

For general graphs, such as social networks, communication networks, or dependency graphs, the heuristic must be derived from problem-specific knowledge. For example, a social network application might use information such as mutual connections, community membership, or other metadata to estimate how *close* a user is to a target user.

In many cases, however, no suitable heuristic is known. In these situations, the heuristic is often defined as

$$h(n) = 0$$

for all nodes n . Substituting this into the A* evaluation function gives

$$f(n) = g(n)$$

which causes A* to reduce to Dijkstra's algorithm. While this guarantees an optimal path, it loses the efficiency gains that a good heuristic can provide.

There is an important property that the Heuristic Function must satisfy:

$$h(n) \leq \text{true remaining cost}$$

When an admissible heuristic is used (such that $h(n) \leq \text{true remaining cost}$), A* is guaranteed to find an optimal path if one exists.

1.3.3 The Total Estimated Distance $f(n)$ and Priority Queues

The purpose of the function

$$f(n) = g(n) + h(n)$$

is to assign a priority to each node during the search. Recall from our discussion of priority queues that each element is associated with a priority value, and the element with the highest (or lowest) priority is removed first.

In A^* , the priority of a node is its estimated total distance $f(n)$. Nodes with smaller values of $f(n)$ are considered more promising because they represent paths that appear likely to reach the goal with a lower total cost.

As with Dijkstra's algorithm, A^* maintains a priority queue containing nodes that have been discovered but not yet fully explored. However, instead of prioritizing nodes solely by their known distance from the start node, A^* prioritizes nodes according to their estimated total distance

$$\text{priority}(n) = f(n)$$

At each step, the node with the smallest $f(n)$ value is removed from the priority queue and explored. When a neighboring node is discovered, its path cost $g(n)$ is updated, a new heuristic estimate $h(n)$ is calculated, and a new value of $f(n)$ is computed.

There are some good references about this in your digital resources, as well as a fun Minecraft-based explanation [here](#).

1.4 Travelling Salesperson Problem and Nearest Neighbour

1.5 Isomorphism

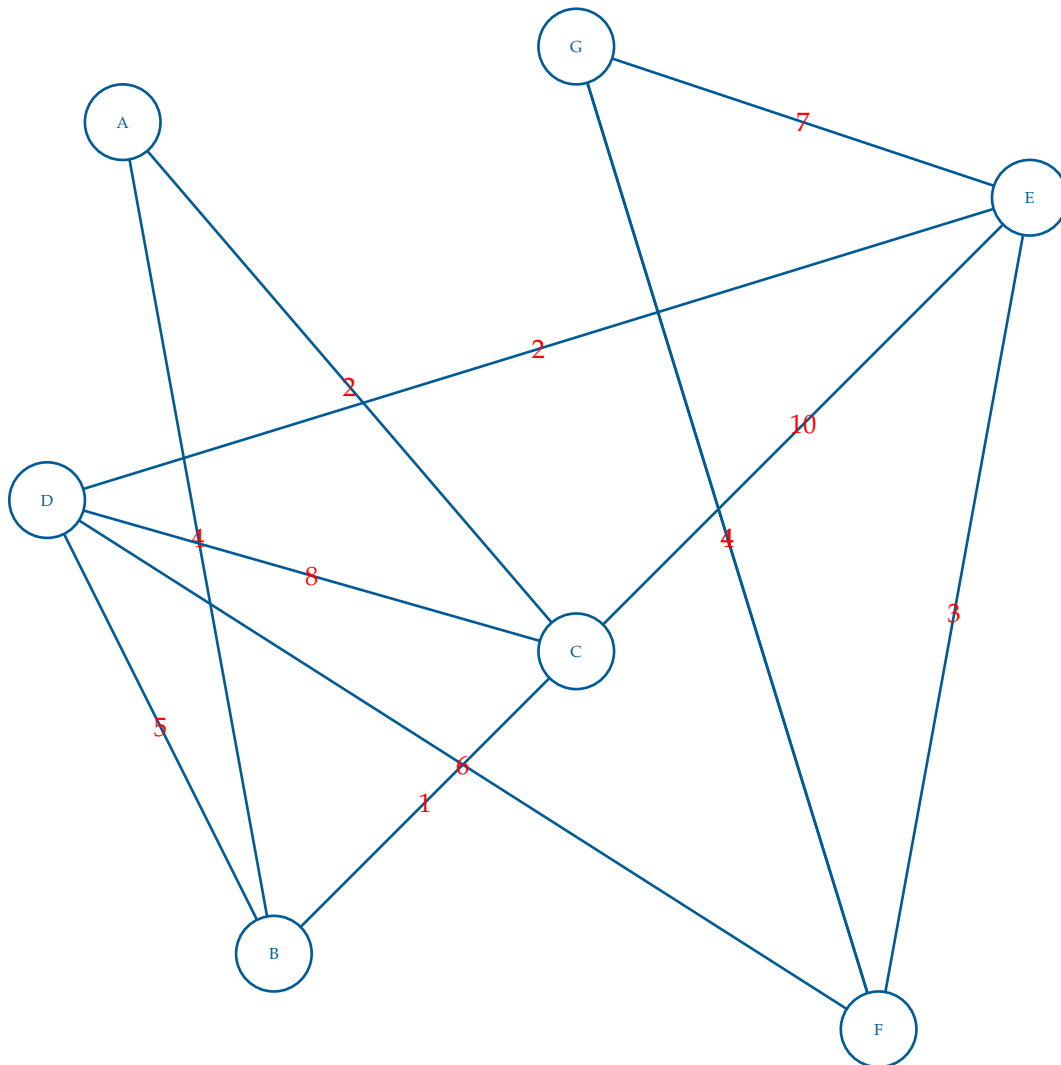
1.6 Topological Sort and DAGs

1.7 Proofs

This is a draft chapter from the Kontinua Project. Please see our website (<https://kontinua.org/>) for more details.

Answers to Exercises

Answer to Exercise 1 (on page 3)





INDEX

A* Algorithm, 4
A-star Algorithm, 4
adjacency matrix, 1

Chebyshev distance, 5

directed adjacency matrix, 2

Euclidean distance, 5

heuristic, 4

matrix, 1

taxicab distance, 5